VORTEX-GEN — FULL PROJECT ARCHITECTURE

1. Executive Summary & Tech Stack

Project Overview

Vortex-Gen is a full-stack aerodynamic simulation platform designed for aerospace engineering students, researchers, and hobbyists. It combines interactive WebGL-based visualization with a Python neural-network backend (NeuralFoil) to deliver real-time fluid dynamics predictions. The recent upgrade introduces a feature-based architecture, an interactive pedagogical Lab Manual, and side-by-side comparative analysis of airfoil geometries.

Technology Stack

Frontend (Vite + React 19 + TypeScript/JSX)

- Framework: React with Vite bundler.
- Styling: Tailwind CSS v4 (Glassmorphism, dark-mode native).
- Animations: Framer Motion (Page transitions, Interactive Lab Manual).
- Data Viz: Recharts (Cl/Cd/Polar charts) & Three.js/React-Three-Fiber (3D particles).
- State: AppContext & Zustand (planned migration).
- Exports: jsPDF & html2canvas for multi-page reports.

Backend (Node.js + Python Daemon)

- Gateway: Express.js (Node.js) handling REST requests & Auth.
- Engine: Python 3 daemon using PyTorch and NeuralFoil.
- IPC Protocol: JSON over stdin/stdout for high-performance process communication.

Infrastructure & Integrations

- Database/Auth: Supabase (PostgreSQL + JWT Auth).
- Payments: Stripe (Tiered Subscriptions).
- Hosting: Vercel (Frontend) & Railway (Backend).
- Emails: Resend API.

VORTEX-GEN — FULL PROJECT ARCHITECTURE

2. Core Features & Architecture Transition

Key Features

1. Academy & Explorer: Interactive learning modules for Fuselage, Wings, and Airfoils.
2. Advanced Wind Tunnel Lab: Real-time simulation dashboard with environmental controls.
3. Interactive Lab Manual: Step-by-step pedagogical walkthroughs using Framer Motion.
4. Side-by-Side Compare Mode: Simultaneous visualization of two airfoils (Metrics & Charts).
5. Intelligent Analytics: Golden Lift optimization, Stall alarms, and Drag Polar analysis.
6. Artifact Generation: 3D STL exports for CAD/printing and multi-page PDF generation.

Architecture Transition (Legacy JSX -> Modular TSX)

The project is currently undergoing a structural migration to a feature-sliced architecture. Historically, the frontend resided in a flat 'pages' and 'components' directory. The new architecture is scoped into distinct bounded contexts within 'src/features/'. This enhances maintainability, testability, and type safety as the application scales.

VORTEX-GEN — FULL PROJECT ARCHITECTURE

3. Directory Structure (File Tree)

```
backend/
|-- run_nf.py               # Python NeuralFoil daemon (stdin/stdout protocol)
|-- server.js               # Express API, auth middleware, Stripe & Resend logic
|-- Dockerfile              # Docker config for Railway deployment
|-- requirements.txt        # Python dependencies
|-- .env.example            # Backend env variable reference
+-- package.json
frontend/
|-- public/                 # Static assets (banner, icons, favicon)
+-- src/
    |-- assets/             # Images, SVGs, and branding assets
    |-- components/         # Shared UI components and global modals
    |-- config/             # Constants and configuration
    |-- context/            # AppContext (global state for Auth/Tier)
    |-- features/           # New Feature-Based Architecture (Next-Gen TS)
    |   |-- academy/        # Educational modules (Airfoil, Fuselage, Wings)
    |   |-- auth/           # Authentication flows
    |   |-- lab/            # Aerodynamic simulation lab core
    |   |-- landing/        # Landing page module
    |   |-- pricing/        # Subscription plans
    |   |-- profile/        # User profile and hangar
    |   +-- settings/       # App settings
    |-- hooks/              # Custom React hooks
    |-- i18n/               # Internationalization (en, ar)
    |-- layouts/            # Page wrappers (DashboardLayout, ExplorerLayout)
    |-- lib/                # Utility functions, API and Supabase clients
    |-- pages/              # Legacy JSX pages (Home, Login, Signup, Explorer)
    |-- services/           # API Services
    |-- store/              # State management stores
    |-- styles/             # CSS Modules, Themes, Animations
    |-- App.jsx / App.tsx
    |-- main.jsx / main.tsx
    +-- index.css
vercel.json                 # Vercel deployment config
start_servers.bat           # Windows: starts frontend + backend together
README.md                   # Project documentation
```

VORTEX-GEN — FULL PROJECT ARCHITECTURE

4. API Architecture & Data Flow

REST API Endpoints (Node.js Gateway)

GET /api/status - Service health check & NeuralFoil daemon status.
POST /api/analyze - Proxies aerodynamics reqs to Python daemon. Validates JWT.
POST /api/increment-import - Tracks user imports, enforces free/pro/promax tier limits.
POST /api/create-checkout-session- Initializes Stripe Hosted Checkout for upgrades.
POST /api/webhooks/stripe - Handles Stripe events (checkout.session.completed).
POST /api/log - Browser error relay for remote debugging.

Data Flow (Simulation Request)

1. Client triggers simulation with shape geometry (points) and environment config.
2. React sends POST /api/analyze to Express backend with Supabase JWT in Authoriza header.
3. Express middleware validates JWT against Supabase. Checks subscription tier.
4. Express converts request to JSON string and writes to Python NeuralFoil daemon's
5. Python process decodes JSON, runs PyTorch NeuralFoil surrogate or NACA analytic
6. Python writes results array back to stdout as JSON.
7. Express parses stdout, streams response back to React frontend.
8. Recharts & Three.js consume data for Cl/Cd charts and 3D particle visualization.

Supabase Architecture Overview

The application utilizes Supabase for Authentication and Data Storage.
Authentication is handled via Email/Password and Google OAuth.

Table: profiles

id (UUID)	Primary Key, References auth.users
display_name (TEXT)	User's full name
tier (TEXT)	Subscription status ('free', 'pro', 'pro_max')
imports_count (INT)	Tracking custom airfoil imports per month
stripe_customer	Stripe Customer ID reference
created_at (TS)	Account creation timestamp

Security:
- Row Level Security (RLS) is enabled on all tables.
- Users can only SELECT and UPDATE their own profile records.
- Backend API relies on the Service Role Key to bypass RLS for webhook updates.